

Take-Structured Decoding of a Delta-Compressed Depth–Infrared Video Ensemble under a 100 MB Budget

CUHK-X Challenge Technical Report — Small Model Track

Team Information

Team name	KTK	Kaggle team name	KTK
Track	Small Model Track	Faculty advisor	N/A
Contact email	kuo.kuant@gmail.com	Team members	Kuan-Ting Kuo

1 Summary

This track asks for 40-class human activity recognition on clips of unseen subjects, with every weight used at inference in one checkpoint below 100 MB [1, 2]. Two R(2+1)D-34 networks [3] classify each clip from person-cropped Depth_Color and infrared (IR) video, read as one pixel-aligned four-channel input, and a linear readout of a frozen VideoMAEv2 ViT-S [4] is blended into their average. The clips were cut from continuous recordings (*takes*) in which no activity occurs twice, and the frame filenames keep the recorder’s clock, so the 405 test clips regroup into 144 takes without any label; each take is decoded as a set of distinct labels under a transition prior, and takes that repeat one routine share evidence. This works because per-clip errors inside a take are often mutually inconsistent — two clips of one take cannot show the same activity — and joint decoding resolves such conflicts. Storing the second network as a lossless delta against the first fits everything into one 97.5 MB file. The system scored 0.83823 on the private leaderboard, 22nd of 326 teams.

2 Method Overview

Figure 1 shows the pipeline in three stages. **(1) Preprocessing** (Section 5.1): a COCO-pretrained SSDLite detector [5, 6] fixes one person window per clip, which cuts Depth_Color and IR into four-channel crops in three views: the window at 144^2 (*det*), a tighter box around the moving region at 144^2 (*motion-in-window*) and the window at 248^2 for the transformer. **(2) Recognition** (Section 3.2): two R(2+1)D-34 members read the 144^2 views, and their average F_0 is blended in log space with q , a linear readout of frozen VideoMAEv2 ViT-S features. **(3) Decoding** (Section 3.3): filename clocks group the test clips into takes and link repeated takes; probabilities are shared along the links, and each take is decoded to distinct labels.

The design follows from three properties of the data. Depth_Color and IR share one pixel

grid, so a single network can read them as one input; a detector trained on natural images finds people in IR far more often than in colour-mapped depth (Section 5.1), so IR places the window for both; and clips of one recording constrain one another’s labels, so labels are decided per recording.

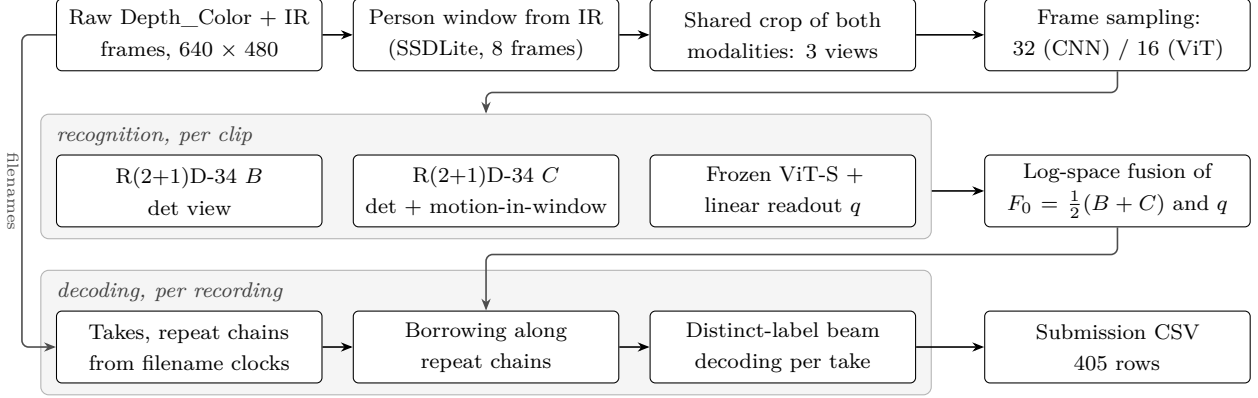


Figure 1: The delivered pipeline. Take and chain structure comes from the frame filenames alone; the fused probabilities enter the decoder at the borrowing step, and each take is decoded once, after borrowing.

3 Modalities and Model Design

3.1 Modalities Used

Modality	Used?	How it is represented	Why it helps
Depth	Yes	Depth_Color PNGs (640×480 , 10 fps), a colour-mapped rendering of depth, kept as RGB in input channels 0–2	Body shape and distance to objects, independent of clothing and lighting
IMU	No	N/A	N/A
mmWave	No	N/A	N/A
Skeleton	Filenames only	Joint coordinates are not used; the filenames are a fallback source of frame timestamps	Supply the recording clock when Depth_Color and IR names are unreadable
Thermal	No	N/A	N/A
Infrared	Yes	8-bit PNGs on the depth sensor’s pixel grid; input channel 3, and the only input of the person detector	Clear silhouette under any lighting

3.2 Architecture and Fusion

Recognition members. B and C are R(2+1)D-34 networks [3] pretrained on IG-65M and Kinetics-400 [7, 8], with weights from `ig65m-pytorch` [9]. The first convolution takes four channels: the pretrained RGB filters initialise the Depth_Color channels and their mean the IR

channel. A member averages its softmax over four passes of its input (Section 5.1): unchanged, flipped horizontally, and with the frame sequence shifted cyclically by one step either way. B reads the det view; C averages its det and motion-in-window predictions; $F_0 = \frac{1}{2}(p_B + p_C)$. The backbone, four-channel input, stem widening, Kinetics normalisation and four-pass averaging follow public notebooks for this competition [10, 11].

Frozen transformer readout. The public VideoMAEv2 ViT-S/16 distilled from ViT-g on Kinetics-710 [4, 12] is a fixed feature extractor: no encoder weight is updated and its 710-class head is discarded. It encodes the Depth_Color and IR inputs separately, each plain and flipped, and its tubelet embedding turns 16 frames of 224^2 pixels into $8 \times 14 \times 14$ tokens of dimension 384. The output tokens are averaged over five regions — the whole grid and its four 7×7 quadrants — and layer-normalised. Each patch is weighted by the fraction of its pixels that lie inside the camera frame (Figure 2f), which reduces to the plain mean when the window lies inside the frame. This yields $I, D \in \mathbb{R}^{5 \times 384}$ per flip for IR and depth. After per-dimension standardisation with training statistics,

$$x = [I, D, I \odot D, |I - D|] \in \mathbb{R}^{7680}$$

($\odot$ and $|\cdot|$ elementwise) is standardised again and mapped to the 40 classes by a multinomial logistic regression (L2 penalty with inverse strength 0.01, L-BFGS) fitted on all 2,931 training clips; averaging the two flips gives q . The head is stored as an FP32 linear layer, so inference needs no scikit-learn.

Fusion. The combination is late and fixed:

$$f = \text{normalize}(\exp(0.8 \log F_0 + 0.2 \log q)), \quad \text{probabilities clipped at } 10^{-12}, \quad (1)$$

on clips whose IR and depth both decoded, and $f = F_0$ otherwise. The geometric blend was preferred to an arithmetic one on out-of-fold accuracy.

3.3 Take-Structured Decoding

Takes. A training clip is stored as `action/user/trial`, where one `user/trial` (trial = scene–take–repetition) is a continuous recording from which the clips of several activities were cut. The path holds one clip per activity and recording, so a recording contains each activity at most once; the order of activities is also regular: over the 791 training takes, knowing the previous activity lowers the entropy of the next from 5.04 to 2.72 bits. Test folders are anonymised, but frame filenames keep the date, a 100 ms timestamp and the recorder’s frame counter, e.g. `Depth_2025-06-11_13-21-46.616_00000067_Color.png`. Subtracting 100 ms $\times$ counter from a clip’s first timestamp gives the recorder’s start time; clips with the same date and start time form a take, ordered by time. This rule, which reads filenames only, recovers all 791 training takes exactly and places the 405 test clips into 144 takes of 1–8 clips (median 2), with no rejected clip and no tie.

Joint decoding. For a take with clips $1, \dots, n$ in recording order and fused probabilities f_i , the decoder chooses pairwise distinct labels $y_1, \dots, y_n$ that maximise

$$\sum_{i=1}^n \log f_i(y_i) + w \sum_{i=2}^n \log T(y_{i-1}, y_i),$$

where T is the 40×40 matrix of activity transitions counted in the training takes with add-one smoothing and $w = 0.75$. Without the prior this is an assignment problem that the Hungarian method solves exactly [13]; with it, distinctness couples all positions while the prior couples neighbours, so we use beam search (64 beams, the 16 most probable classes per clip). The formulation is close to set-constrained decoding for action segmentation [14], with the constraint recovered from metadata instead of supervision.

Repeat chains. Subjects record a routine several times in a row (the repetition digit of the trial code), with the activities in the same order each time. Consecutive test takes of the same date are therefore linked when the later one starts less than 120 s after the start of the earlier one’s last clip and both have the same number of clips, and position k of every take in a chain is treated as the same activity. Each take’s log-probabilities are increased by $\lambda = 0.25$ times the sum of the other takes’ log-probabilities at the same position, renormalised per clip and decoded as above. On complete training takes this rule forms 356 links, 97.8% of them between repetitions of one trial and 95.2% between identical activity sequences, and adds 1.7 pp for the delivered configuration. On the test set it forms 39 chains (35 pairs, 4 triples) over 235 clips and changes 10 of the 405 answers.

Fallbacks. Below 50% take coverage, for instance with filenames that carry no clock, decoding and borrowing are skipped and each clip takes its most probable class (the Raw columns of Table 1), as do clips outside any take. A wrong grouping that passes these checks goes undetected and can lower accuracy.

4 Data Sources and How They Are Used

4.1 Data Source Usage Table

Data source	Type	Which part	Where it enters	Training only or both?
Official challenge data	Official	Depth_Color, IR; Skeleton filenames	Training of every component; preprocessing and decoding	Both
R(2+1)D-34, IG-65M→K400 [7, 9]	Pretrained	Full backbone	Initialisation of B , C and one pseudo-label teacher	Training only
ir-CSN-152, IG-65M→K400 [15, 16]	Pretrained	Full backbone	Initialisation of the other pseudo-label teacher	Training only

Data source	Type	Which part	Where it enters	Training only or both?
VideoMAEv2 ViT-S, K710 [4, 12]	Pretrained	Encoder	Frozen feature extraction	Both
SSDLite320-MobileNetV3, COCO [5, 17]	Pretrained	Full detector	Person window	Both
Test-set pseudo-labels	Official, unlabelled	405 test clips, soft labels	Fine-tuning of B	Training only

4.2 Open Questions on Data Usage

Splits and external sources. Of the 3,036 training clips from 18 subjects, the 2,931 with Depth_Color and IR form the training pool, on which every delivered component is trained; local validation uses four subject-disjoint folds of it (760/843/629/699 clips, 5/5/4/4 subjects; Section 5.2). Beyond the four public checkpoints above, no external dataset, API, retrieval corpus or knowledge base is used.

Indirect use. *Shared initialisation:* four R(2+1)D-34 models, each trained for 60 epochs on three of the four folds, were averaged uniformly [18] and their BatchNorm statistics re-estimated on 1,600 training clips; B and C are fine-tuned from this average, through which every training subject reaches both. *Pseudo-labels:* B is fine-tuned on the 2,931 labelled clips plus the 405 test clips, whose targets are the mean four-pass softmax (temperature 1) of two teachers trained on labelled clips only — the R(2+1)D-34 average and an average of four ir-CSN-152 fold models [15] (60 epochs, 224^2 crops of the 248^2 view). A test clip has loss weight 0.5 against 1 for a labelled clip, so the pseudo-labels carry $0.5 \times 405/2931 = 6.9\%$ of the labelled loss weight; there is no confidence threshold and no manual annotation. The organisers confirmed that self-training on test clips is permitted. C uses no test clip and no teacher.

Subsetting and cleaning. Four test clips ship IR files without image data; their IR channel is set to zero and they keep $f = F_0$ (Equation 1).

Augmentation and recipe. Training uses random 128^2 crops of the 144^2 view, a random frame in each temporal segment, horizontal flip with $p = 0.5$, random erasing [19] with $p = 0.25$, mixup [20] with $\alpha = 0.2$ and label smoothing [21] of 0.1 on labelled clips. None of these is used at inference, which takes centre crops, segment centres and the four passes of Section 3.2. Both members are fine-tuned for 10 epochs at batch size 8 with AdamW [22] and a one-cycle schedule [23] (15% warm-up, peak learning rate 3×10^{-5} for the classifier and 3×10^{-6} for the backbone), weight decay 0.01, dropout of 0.3 on the backbone output and 0.5 before the classifier, mixed precision and seed 42. B trains on the det view, C on the det or motion-in-window view drawn per clip and epoch. Finally the BatchNorm statistics of both are re-estimated on 1,600 training clips without gradient steps [24].

Inference-time sources. Only the test data and the checkpoint; the runtime refuses any other weight file and disables network connections.

5 Training, Validation, and Expected Generalization

5.1 Input Construction and Preprocessing

Expected input. Per clip, `Depth_Color/` and `IR/` PNG sequences (640×480 , 10 fps) with matching filenames, and `Skeleton/`, of which only the filenames are read. Test clips have 2–101 frames (median 20, about 1.9 s). No preparation is needed before a run: `inference.sh` performs the steps below from the raw folders, and Figure 2 follows one clip through them.

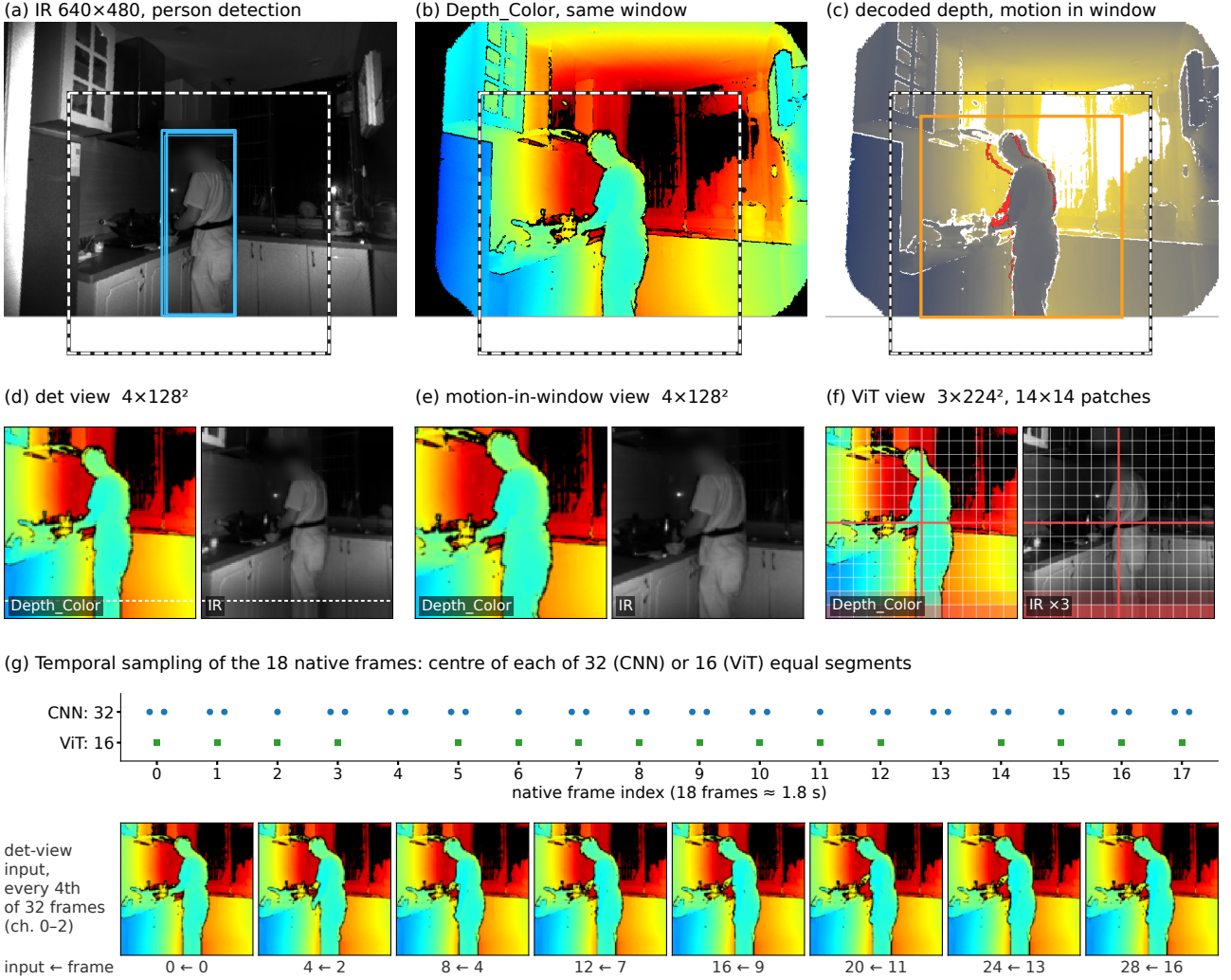


Figure 2: One clip through the preprocessing steps of Section 5.1, rendered with the delivered code and checkpoint (training clip `9_Pour_drinks/user17/5-2-1`, 18 frames; faces blurred in this figure only).

1. **Frames.** Clips longer than 32 frames keep 32 evenly spaced frames (17.8% of test clips).
2. **Person window.** SSDLite runs on eight evenly spaced IR frames and keeps the best person box of each (Figure 2a, blue). The boxes are accepted if one scores at least 0.30, or if at least four score at least 0.10 with centres scattered by less than 10% of the frame width. Otherwise the detector reruns on four overlapping corner tiles (60% of the frame’s width and height) after a 1st–99.5th percentile contrast stretch; then a motion box on decoded depth is tried, and finally the whole frame. The window is a square on the median box centre with side

$\max(1.4 \times \text{largest box side}, 0.35 \times \text{frame width})$, not clipped to the frame (Figure 2a, dashed; this one extends 60px below the frame). This geometry follows Desale [11], with YOLO11n replaced by torchvision’s SSDLite [17]. On the test set 385 clips pass on a confident box, 11 on agreement, 5 on tiles and 4 on the motion box. On 60 test clips, one frame each, the detector finds a person in 96.7% of IR frames and in 16.7% of Depth_Color frames.

3. Depth decoding. A lookup table orders the 254 palette colours along the jet colour map and maps each to a rank from 0 to 254, monotone in depth but not metric; black and unknown colours mean no return (Figure 2c, white). Decoded depth serves only the motion boxes; the networks see the Depth_Color RGB values. The table is stored in the checkpoint.

4. Shared crop and views. Both modalities are cut with the same window and frame indices (Figure 2b); parts outside the frame repeat the edge pixels, which keeps the window square. The motion-in-window view uses a tighter box: the pixels in the top 2% of motion — the mean absolute deviation of decoded depth from its per-pixel temporal median — inside the person window (Figure 2c, red), with a 10% margin, squared (orange); if this box degenerates, the person window is used. The det and motion-in-window crops are resized bilinearly to 144^2 and the transformer crop to 248^2 , and frames are stored as uint8 with channels Depth_Color R, G, B and IR. Figures 2d and 2e show the member inputs; below the dotted line in 2d the window lies outside the frame and repeats the last real row. Figure 2f shows the transformer input with its 14×14 patches, the quadrant boundaries (red) and the patches whose support weight is below one (shaded).

5. Frame sampling. The stored frames are split into K equal segments and the centre frame of each is taken [25], with $K = 32$ for the members and $K = 16$ for the transformer; short clips repeat frames (Figure 2g, with every fourth of the 32 det-view input frames).

6. Network tensors. Pixels are divided by 255 and normalised with the Kinetics mean and standard deviation for the members (their average for IR), stored in the checkpoint, and with the ImageNet constants for the transformer. A member thus reads a $4 \times 32 \times 128 \times 128$ tensor (centre 128^2 of the 144^2 view) per clip and view, and the transformer a $3 \times 16 \times 224 \times 224$ tensor (centre 224^2 of 248^2 , IR copied to three channels) per modality and flip.

7. Alignment. Depth_Color and IR share the sensor, the pixel grid and the frame filenames, so no registration is needed.

5.2 Validation Strategy and Expected Generalization

Question	Details
Local validation split type	Cross-subject: four subject-disjoint folds
Held-out subjects used locally	5/5/4/4 per fold, 18 in total
Held-out environments used locally	N/A; the release does not label environments
Local validation result	0.7999 on test-like partial takes, 0.8092 on complete takes (four-fold means after decoding; Table 1)
Public leaderboard result	0.84079

Question	Details
Final (private) leaderboard result	0.83823, 22nd of 326 teams
Gap between local and public result	Public +4.1 pp and private +3.8 pp over the test-like local result
Cross-subject result, if tested	The local protocol is cross-subject (above)
Expected performance on unseen subjects	About 0.80 with the recording protocol unchanged; about 0.77 without take structure
Expected performance in a new environment	Lower; Section 9 lists where the system breaks first
Shift that worries you most	A different recording protocol: no clock in filenames, activities repeated within a recording, or routines that are not repeated

A weight-averaged model cannot be scored on folds that trained any of its parts, so every component is retrained per fold with that fold’s subjects held out, and no held-out label reaches any component of its fold:

Component	Delivered system	Validation fold k
Starting weights of B , C	Average of the four fold models	Fold model k (trained without fold k)
B , labelled clips	All 2,931	Clips outside fold k
B , pseudo-labelled clips	The 405 test clips	The test-like subset of fold k , without labels
B , pseudo-label teachers	R(2+1)D-34 and ir-CSN-152 averages	R(2+1)D-34 and ir-CSN-152 fold models k
C and the readout head	All 2,931	Clips outside fold k
Transition prior	All 791 training takes	Takes outside fold k

Complete takes decode every held-out recording; the *test-like* population keeps one contiguous run per held-out take, with lengths drawn from the test take sizes (1,631 clips), and is the set on which fold k ’s B was self-trained, as the delivered B was on the test clips. The folds were reused across many comparisons, so they rank variants rather than predict test scores. The recognition part is ordinary cross-subject transfer, which the folds measure directly, although 18 subjects is a small sample; the decoder’s gain holds only as long as the recording protocol does.

6 Results and Evidence

Table 1 gives four-fold accuracy before decoding (Raw) and after take decoding without repeat chains (Decoded); Table 2 isolates repeat-chain borrowing and the transformer readout on the leaderboard.

Take decoding contributes the most: it adds 2.7–2.9 pp on test-like takes and 5.0–5.3 pp on complete takes, where each take constrains more clips, for every recognition variant. Repeat-chain borrowing adds one to three answers on both splits for both recognisers; each run also

writes the answers before borrowing, so the ten changed rows can be inspected. The readouts add 0.8–1.5 pp on the folds and differ from one another by less than the fold-to-fold spread; on the leaderboard the readout moves the score by between three answers down and one up. As a failure mode, clips with unreadable IR fall back to F_0 (Section 4.2), so a systematic IR failure would reduce the system to F_0 without any error.

Table 1: Four-fold cross-subject accuracy (mean over the subject-disjoint folds). *Global*: one mean-pooled region, inverse L2 strength 1; *region*: the five regions of Section 3.2; *weighted*: with support weighting. Arithmetic and geometric refer to the blend with F_0 . Bold: the delivered configuration.

Recognition	Test-like takes		Complete takes	
	Raw	Decoded	Raw	Decoded
F_0 (members $B+C$)	0.7576	0.7861	0.7434	0.7965
F_0 + global readout, arithmetic	0.7667	0.7939	0.7514	0.8046
F_0 + region readout, geometric	0.7718	0.8008	0.7571	0.8074
F_0 + weighted region readout, geometric	0.7712	0.7999	0.7564	0.8092

Table 2: Leaderboard ablation. All rows use the delivered members and take decoding; the last two use the delivered checkpoint. One answer is 0.50 pp on the public split (201 clips) and 0.49 pp on the private split (204 clips).

Recognition	Repeat borrowing	Public	Private
F_0	no	0.83084	0.83823
F_0	yes	0.83582	0.84313
F_0 + readout	no	0.83084	0.82352
F_0 + readout (delivered)	yes	0.84079	0.83823

7 Efficiency and Reproducibility

7.1 Training and Inference Cost

Item	Details
Training hardware	One NVIDIA GeForce RTX 5070 Ti
Approximate training time	Fold models: R(2+1)D-34 about 8 h, ir-CSN-152 7.8 h; B 1,728 s (plus 75 s pseudo-labels, 50 s BatchNorm); C 1,613 s; transformer features 5 min; readout 11 s
Approximate training cost or GPU hours	About 17 GPU-hours, mostly the fold models; development runs not counted
Inference hardware	One CUDA GPU; there is no CPU path
Approximate inference speed	405 clips in about 360 s from the raw folders (0.89 s per clip), of which 128 s (0.32 s per clip) is network computation
Peak memory	2.27 GiB allocated, 4.04 GiB reserved (PyTorch CUDA allocator)

7.2 Artifact Access and Reproducibility

Item	Details
Artifact access mode	Public
Code, checkpoint and run guide	https://github.com/kuokuant-oss/CUHK-X-Competition-Small-Model-Track : one repository with the code, the report, weights/model.pt (97,478,875 bytes, SHA256 in checksums.txt), README.md, METHOD.md and TRAINING.md
Report files	CUHKX_TechnicalReport_SmallModelTrack.pdf; LaTeX source CUHKX_TechnicalReport_source.zip; no supplementary PDF
Inference entry script	run_inference.sh, which calls inference.sh
Required input directory	The folder holding SM_test_0001 ... SM_test_0405, each with Depth_Color/, IR/ and Skeleton/
Final config file	configs/paths.yaml (paths only); the model configuration is stored in the checkpoint
Key hyperparameters for reproduction	Readout: support-weighted regions, geometric blend with weight 0.2 on clips with IR and depth (--active-readout S1-G , set in inference.sh); decoder: 64 beams, 16 classes per clip, transition weight 0.75; repeat chains: gap below 120s, equal clip count, $\lambda = 0.25$
Random seed or deterministic setting	Seed 42; single-threaded BLAS; TF32 and cuDNN benchmarking disabled
Post-processing or ensemble rule	Equation 1, then repeat-chain borrowing, then take-level beam decoding
Special dependency or environment note	Install PyTorch from the CUDA 12.8 wheel index before requirements.txt
CUDA / package version note	Python 3.12.14; torch 2.11.0+cu128, torchvision 0.26.0+cu128, pytorchvideo 0.1.5, timm 0.6.13, numpy 2.5.2, pandas 3.0.5
What changed after the public phase	Nothing; checkpoint and decoder are those of the scored submission (Kaggle submission 56254406)

Minimum steps to reproduce. Clone the repository, install PyTorch from the CUDA 12.8 index and then **requirements.txt**, check the files and run:

```
shasum -a 256 -c checksums.txt
./run_inference.sh --data_dir <TEST_DIR> --sample_submission <SAMPLE_CSV> \
--output submission.csv
```

A correct run writes 405 rows and prints the CSV and prediction-vector hashes next to the expected values.

8 Small Model Track

Item	Details
Final model size	97,478,875 bytes (97.48 MB, 92.96 MiB), one file
Parameter count	152,905,688 stored values: 63,590,251 per R(2+1)D-34 member, 21,921,792 ViT-S encoder, 3,473,074 detector, 330,320 readout (307,240 linear-layer parameters, 23,040 standardisation statistics, a 40-entry class map)
Quantization or compression	int8 with per-output-channel scales for both members and the encoder; member C stored as a lossless difference against B ; LZMA or zlib per tensor and zlib over the file; per-tensor SHA256 checked on load
Target deployment hardware	One CUDA GPU with at least 8 GB of memory
Latency on target hardware	0.89 s per clip including preprocessing; 0.32 s per clip for the networks
Memory footprint	2.27 GiB peak allocated at batch size 8

The four networks hold 152.9 M values, beyond 100 MB at FP32 or FP16. Int8 quantisation with per-output-channel scales [26, 27] and entropy coding bring B to 51.3 MB and the encoder to 19.9 MB, leaving no room for a second member stored alike. Since B and C share architecture and starting weights, C is stored as the mod-256 difference of its int8 codes from B ’s (for three tensors, from B ’s codes rescaled to C ’s scales), which compresses to 12.8 MB and is restored exactly, checked against a stored SHA256. With the detector (11.6 MB, FP32) and the readout (1.4 MB) the weights take 97.0 MB; per-tensor SHA256 manifests, metadata and serialisation add 1.9 MB, and an outer zlib layer compresses the resulting 98.9 MB to 97.48 MB, 2,521,125 bytes under the limit. The runtime checks the size before any work. The budget thus buys a second backbone and a second view of the person instead of one larger network.

9 Limitations and Lessons Learned

What did not work. A thermal-video member at weight 0.2 raised out-of-fold accuracy by 0.6–0.9 pp but lowered the public score (0.82587 to 0.81094; private unchanged at 0.82352).

Most fragile assumption. That test filenames keep the recording clock and that a recording contains each activity at most once. Missing clocks are caught by the coverage check, which switches to per-clip prediction (Section 3.3); a recording that repeats an activity, or clips placed in the wrong take, pass every check and can lower accuracy unnoticed.

Where it breaks first. The person detector, if camera placement or subject scale changes; the depth colour map, if it differs from the stored palette, since unknown colours decode as no return; then the recognition members, on subjects unlike the 18 training subjects.

Validation. Repeat borrowing rests mainly on the leaderboard: on test-like folds only 69% of its links join identical activity sequences and its local gain is 0.1–0.2 pp. Close variants

ranked differently on the two splits (Table 2), so a single public score is a noisy basis for choosing between them.

With one more week. Same-modality pretraining, e.g. on the depth and infrared streams of NTU RGB+D 120 [28], and estimating take structure from the data so that decoding degrades gradually when the protocol is only partly kept.

References

- [1] Siyang Jiang, Mu Yuan, Xiang Ji, Bufang Yang, Zeyu Liu, Lilin Xu, Yang Li, Yuting He, Liran Dong, Wenrui Lu, Zhenyu Yan, Xiaofan Jiang, Wei Gao, Hongkai Chen, and Guoliang Xing. A large-scale multimodal dataset and benchmarks for human activity scene understanding and reasoning. In *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys '26)*, pages 352–370. ACM, 2026. doi: 10.1145/3745756.3809209.
- [2] CUHK-X Competition Small Model Track. Kaggle competition, 2026. <https://www.kaggle.com/competitions/cuhk-x-competition-small-model-track>.
- [3] Du Tran, Heng Wang, Lorenzo Torresani, et al. A closer look at spatiotemporal convolutions for action recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. arXiv:1711.11248.
- [4] Limin Wang, Bingkun Huang, Zhiyu Zhao, et al. VideoMAE V2: Scaling video masked autoencoders with dual masking. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv:2303.16727.
- [5] Andrew Howard, Mark Sandler, Grace Chu, et al. Searching for MobileNetV3. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1905.02244.
- [6] Tsung-Yi Lin, Michael Maire, Serge Belongie, et al. Microsoft COCO: Common objects in context. *arXiv preprint arXiv:1405.0312*, 2014.
- [7] Deepti Ghadiyaram, Matt Feiszli, Du Tran, et al. Large-scale weakly-supervised pre-training for video action recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. arXiv:1905.00561.
- [8] Will Kay, Joao Carreira, Karen Simonyan, et al. The Kinetics human action video dataset. *arXiv preprint arXiv:1705.06950*, 2017.
- [9] MoabitCoin. ig65m-pytorch: PyTorch 3D video classification models pre-trained on 65 million Instagram videos. GitHub repository, 2019. <https://github.com/moabitcoin/ig65m-pytorch>.
- [10] Fusu (phuongncn). LB 0.711 | YOLO person crop + R2Plus1D <100MB. Kaggle notebook, 2026. <https://www.kaggle.com/code/phuongncn/lb-0-711-yolo-person-crop-r2plus1d-100mb>.
- [11] Kunal Desale. YOLO for CUHK-X. Kaggle notebook, version “LB 0.716 → v9”, 2026. <https://www.kaggle.com/code/kunaldesale2408/yolo-for-cuhk-x>.
- [12] OpenGVLab. VideoMAE V2: code and model zoo. GitHub repository, commit 29eable; checkpoint vit_s_k710_dl_from_giant, 2023. <https://github.com/OpenGVLab/VideoMAEv2>.
- [13] H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2): 83–97, 1955. doi:10.1002/nav.3800020109.
- [14] Jun Li and Sinisa Todorovic. Set-constrained Viterbi for set-supervised action segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. arXiv:2002.11925.
- [15] Du Tran, Heng Wang, Lorenzo Torresani, and Matt Feiszli. Video classification with channel-separated convolutional networks. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1904.02811.
- [16] MMLab Contributors. OpenMMLab’s next generation video understanding toolbox and benchmark. GitHub repository, ir-CSN-152 IG-65M→Kinetics-400 checkpoint, 2020. <https://github.com/open-mmlab/mmdetection>.
- [17] TorchVision maintainers and contributors. TorchVision: PyTorch’s computer vision library. GitHub repository, 2016. <https://github.com/pytorch/vision>.
- [18] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2203.05482.
- [19] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *AAAI Conference on Artificial Intelligence*, 2020. arXiv:1708.04896.
- [20] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.09412.
- [21] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the Inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. arXiv:1512.00567.
- [22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.
- [23] Leslie N. Smith and Nicholay Topin. Super-Convergence: Very fast training of neural networks using large learning rates. *arXiv preprint arXiv:1708.07120*, 2017.
- [24] Yanghao Li, Naiyan Wang, Jianping Shi, Jiaying Liu, and Xiaodi Hou. Revisiting batch normalization for practical domain adaptation. *arXiv preprint arXiv:1603.04779*, 2016.
- [25] Limin Wang, Yuanjun Xiong, Zhe Wang, et al. Temporal Segment Networks: Towards good practices for deep action recognition. In *European Conference on Computer Vision (ECCV)*, 2016. arXiv:1608.00859.
- [26] Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1510.00149.
- [27] Bull (welshonionman). [LB0.667] baseline with YOLO person crop. Kaggle notebook, 2026. <https://www.kaggle.com/code/welshonionman/lb0-667-baseline-with-yolo-person-crop>.
- [28] Jun Liu, Amir Shahroudy, Mauricio Perez, et al. NTU RGB+D 120: A large-scale benchmark for 3D human activity understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv:1905.04757.